

Fitness Hub

A personal fitness command center — workouts, nutrition, sleep, supplements, and steps in one place, with an AI coach on top.

Status	Planning document, v2
Initial target	Responsive web app (mobile later)
Cost target	\$0/month on free tiers

What changed in v2

- 1. Lyfta has a real public API — direct sync, no manual exports.
- 2. Strava added as a primary data source for runs/rides/swims via OAuth.
- 3. Architecture restructured around a **provider abstraction layer** so Lyfta can be swapped for native in-app workout tracking later without rewriting the rest of the app.

1. What we’re building

A single web app that becomes the source of truth for your fitness life. The core loop:

- Data flows in from external services (Lyfta workouts, Strava runs, Google Fit steps) and your own logging (nutrition, sleep, supplements).
- The dashboard surfaces trends, streaks, and personal records.
- An AI coach reviews recent data on demand and weekly, suggesting adjustments to training, nutrition, recovery, and supplementation.
- An accountability system (streaks + weekly score + optional email nudges) keeps you on the rails.

2. Integration reality check

Where each data source actually stands today. This drives the build order.

Source	What it gives you	Status	Plan
Lyfta	Workouts, sets, reps, weight, est. PRs	Public REST API, no OAuth, no auth, 5k/day	Primary strength source
Strava	Runs, rides, swims, hikes — GPS, pace, HR, power, 1k/day	OAuth 2.0, no auth, 1k/day	Single OAuth source, primary cardio source
Google Fit	Daily steps, distance, active minutes, sleep (Android)	Public OAuth (Android) free	Direct OAuth sync
MyFitnessPal	Nutrition (calories, macros, micros)	Public API shut down years ago	Build native nutrition on Open Food Facts; one
Sleep	Bedtime, wake time, duration, quality	Manual + Google Fit (Android)	Manual primary; auto-fill from Google Fit
Supplements	What, dose, frequency, daily intake—		Native module

MyFitnessPal tradeoff: rather than fight a dead API, building nutrition natively on top of Open Food Facts (free, 3M+ products, barcode lookup) ends up *better* — you own the data, no rate limits, no scraping. Import MFP history once via their CSV export.

Strava heads-up — Single Player Mode: all new Strava apps start limited to 1 athlete. For a personal app that's fine. If you ever want to share it, you submit a form for an athlete-cap increase. Worth knowing now, no action needed.

3. AI model choice (free tier)

Pick: Google Gemini 2.5 Flash via Google AI Studio

Why	Detail
Real, indefinite free tier	1,500 requests/day, 15 RPM, no credit card
Frontier-class quality	Within ~5% of GPT-4o on most tasks
1M token context	Can feed weeks of training + nutrition in one shot
Easy switch later	All AI calls behind a single abstraction — swap to Claude with one file change

Runners-up: Groq (Llama 3.3 70B, 300+ tokens/sec — fastest free option for chat-style coach), OpenRouter (~30 free models behind one API key — useful as fallback).

4. Architecture — designed to outgrow Lyfta

The key design decision in v2: **nothing in the app talks to Lyfta directly** except one adapter file. Everything else talks to an abstract *workout provider* interface. The day you want to ditch Lyfta and build native tracking, you write a new adapter, flip a switch, and the dashboard / PRs / AI / accountability all keep working unchanged.

The provider interface

Every workout source — Lyfta today, native tomorrow, maybe Hevy or Strong later — implements the same interface:

```
interface WorkoutProvider {
  listWorkouts(since: Date): Promise<Workout[]>
  getWorkout(id: string): Promise<WorkoutDetail>
  listExercises(): Promise<Exercise[]>
  getExerciseProgress(exerciseId: string, days: number): Promise<ProgressPoint[]>
  getProviderName(): string
  getCapabilities(): ProviderCapabilities
}
```

Internally, the app uses its own canonical **Workout**, **Exercise**, and **Set** types. Each adapter translates the provider's shape into ours.

What this looks like in practice

Layer	Today (Lyfta phase)	After migration (native phase)
UI / dashboard	Reads canonical Workout[]	Reads canonical Workout[] — unchanged
PR detection	Works on canonical sets	Works on canonical sets — unchanged
AI coach prompts	Built from canonical data	Built from canonical data — unchanged
Workout source	LyftaAdapter calls /api/v1/workouts	NativeAdapter reads from local DB tables
Sync job	Pulls from Lyfta API nightly	No-op (data already local)
Exercise library	Cached from Lyfta /exercises	Owned by you (seeded from Lyfta cache initially)

The migration path

1. Phase A (now): LyftaAdapter is the only workout provider. App is fully functional.
2. Phase B (later): Build native logging UI — sets, reps, rest timers, exercise picker. Write to the same canonical tables the LyftaAdapter writes to. Now both sources coexist.
3. Phase C: Add a NativeAdapter that reads directly from those local tables. Flip the user's active provider from "lyfta" to "native" in settings. Lyfta sync becomes optional (you might still want it for the exercise library or to import past workouts).
4. Phase D: Optionally retire the Lyfta adapter entirely. Or keep it around as an import-only tool.

Crucially: you never have to migrate data. Lyfta-synced workouts are already in your canonical DB. Native-logged workouts go to the same tables. They coexist in the same history.

5. Tech stack

Layer	Choice	Why
Frontend	Next.js 14 (App Router), TypeScript, Tailwind	Easy mobile reuse later (React Native / Expo)
UI	shadcn/ui + Recharts	Clean defaults, fully customizable, free
Backend	Next.js API routes + tRPC	One repo, no separate server
Database	PostgreSQL via Supabase	500MB free + auth + RLS + storage
Auth	Supabase Auth (email + Google)	Google sign-in handy for Google Fit
Hosting	Vercel	Free for personal use, zero-config Next.js
AI	Gemini 2.5 Flash via @google/generativeai	See section 3
Background jobs	Vercel Cron / Supabase Edge Functions	Nightly syncs + weekly AI review
Email	Resend	3k free emails/month for weekly review

Total monthly cost at personal-use volume: **\$0**.

6. Database schema

Designed so the workout tables are **source-agnostic**. The *source* column tracks where a workout came from, but the structure is the same whether Lyfta synced it or you logged it natively.

```
users
  id, email, name, weight_unit, timezone,
  active_workout_provider ('lyfta' | 'native'), created_at

api_credentials
  user_id, provider ('lyfta' | 'strava' | 'google_fit'),
  access_token (encrypted), refresh_token (encrypted),
  expires_at, last_sync_at

workouts                                     ← canonical, source-agnostic
  id, user_id, source ('lyfta' | 'native'),
  source_id (lyfta workout id, nullable for native),
  title, performed_at, body_weight, total_volume,
  duration_seconds, raw_payload (jsonb, nullable)

workout_exercises
  id, workout_id, exercise_id, exercise_name,
  exercise_type, order_index

workout_sets
  id, workout_exercise_id, set_number,
  weight, reps, rir, duration, distance,
  is_completed, set_type, record_type

exercises                                   ← shared library
  id, source ('lyfta' | 'native'), external_id,
  name, image_url, exercise_type,
  equipment_ids, body_part_ids,
  target_muscle_ids, synergist_muscle_ids

cardio_activities                           ← from Strava
  id, user_id, strava_activity_id (unique),
  type ('run'|'ride'|'swim'|'hike'|'walk'|...),
  start_time, duration_seconds, distance_m,
  avg_pace, avg_hr, max_hr, avg_power,
  elevation_gain_m, calories,
  polyline (gps), raw_payload (jsonb)

personal_records                           ← derived
  user_id, exercise_id, record_type ('1rm'|'volume'|'reps'|'pace'|'distance'),
  value, achieved_at, workout_id (nullable), cardio_activity_id (nullable)

daily_steps                                ← from Google Fit
  user_id, date, step_count, distance_meters,
  active_minutes, calories_burned

sleep_logs
  id, user_id, date, bedtime, wake_time,
  duration_minutes, quality_rating (1-5),
  notes, source ('manual' | 'google_fit')
```

```

food_items                                ← Open Food Facts + custom
  id, barcode, name, brand, serving_size,
  calories, protein_g, carbs_g, fat_g,
  fiber_g, sugar_g, sodium_mg, source

nutrition_logs
  id, user_id, date, meal, food_id, servings, logged_at

supplements
  id, user_id, name, dose, unit,
  frequency_per_day, notes, active, created_at

supplement_logs
  id, user_id, supplement_id, taken_at, dose_taken

body_measurements
  id, user_id, date, weight, body_fat_pct,
  chest, waist, hips, biceps, thighs, notes

progress_photos
  id, user_id, date, photo_url, pose, notes

water_logs
  id, user_id, date, amount_ml, logged_at

mood_logs
  id, user_id, date, mood (1-5), energy (1-5),
  stress (1-5), notes

habit_streaks                            ← derived
  user_id, habit_type, current_streak,
  longest_streak, last_completed_date

ai_reviews
  id, user_id, period_start, period_end,
  model_used, prompt, response, created_at

accountability_score                      ← derived weekly
  user_id, week_start, workouts_score,
  cardio_score, nutrition_score, sleep_score,
  supplements_score, total_score, notes

```

7. Feature breakdown by module

A. Strength training (Lyfta sync, native-ready)

User pastes Lyfta API key once. Nightly job pulls from /api/v1/workouts incrementally. Exercise library cached. /api/v1/exercises/progress used directly for charts (Lyfta already calculates est. 1RM). All writes go through the LyftaAdapter → canonical tables, so the rest of the app never knows where the data came from.

B. Cardio (Strava OAuth)

Click 'Connect Strava' → OAuth consent → tokens stored encrypted. Webhook subscription for near-realtime new activities, plus nightly backfill. Display: weekly mileage, pace trend, elevation, route maps via Strava's polyline.

C. Personal records

Auto-detected on every sync — heaviest weight, best volume, best reps for lifts; fastest 5K/10K/half/full, longest run, biggest elevation day for cardio. 'New PR' badge on dashboard.

D. Steps (Google Fit)

OAuth, refresh-token-aware sync. Daily step ring, 7-day average, monthly heatmap.

E. Nutrition (native, replacing MFP)

Search via Open Food Facts (free, no key). Barcode lookup (webcam on desktop, camera on mobile). Meal logging. Daily macro rings, weekly trend. One-time CSV import from MFP.

F. Sleep

Manual log: bedtime + wake + quality. Auto-fill from Google Fit if available. Duration trend, bedtime drift, sleep debt.

G. Supplements

Add supplements with dose + target frequency. Quick-tap 'Taken' buttons. Daily compliance % per supplement. Browser push reminders.

H. Body measurements + photos

Weekly weigh-in prompt. Optional measurements. Progress photo upload with side-by-side comparison. Photos in Supabase Storage (1GB free).

I. Water tracking

Quick +250ml / +500ml buttons, daily goal ring.

J. Mood / energy check-in

30-second daily form: mood, energy, stress sliders. Trends correlated with sleep + load.

K. AI coach

On-demand chat ('How's training looking?') + automatic weekly review. Last 14 days of all data fed to Gemini as structured context. Model returns JSON with summary / wins / concerns / recommendations. Suggestions stored — you mark them 'tried' / 'didn't help' so the AI learns over time.

L. Habit streaks + accountability score

Tracks 6 habits: workout logged, cardio session, calories logged, sleep logged, supplements taken, water goal. Weekly score 0–100. Optional Sunday-night email with score + AI review.

8. Build phases

Each phase is shippable on its own — you can start using the app as you build.

Phase	Duration	What ships
Phase 1 — Foundation	1–2 weeks	Next.js + Supabase + Tailwind + shadcn scaffold. Auth (email + Google). Empty dashboard.
Phase 2 — Strength (Lyfta)	2–3 weeks	Lyfta API key setup screen. LyftaAdapter implementation. Sync job + workout list. PWA install banner.
Phase 3 — Cardio (Strava)	1 week	Strava OAuth flow with token refresh. Activity sync (webhook + nightly fallback). Cardio history.
Phase 4 — Steps (Google Fit)	3–4 days	Google Fit OAuth, daily steps sync, step ring + heatmap.
Phase 5 — Nutrition	1–2 weeks	Open Food Facts search + barcode. Meal logging UI. Macro rings + targets. MFP CSV import.
Phase 6 — Sleep, supplements, water	1 week	Mostly forms + charts. Fast to build.
Phase 7 — AI coach	1 week	Gemini integration with abstraction layer. ‘Review my week’ button. Structured output.
Phase 8 — Accountability	1 week	Habit streak engine. Weekly score. Resend email pipeline + Vercel Cron.
Phase 9 — Body comp + mood check-in	1–2 weeks	Measurements + photo comparisons. Mood check-in. Onboarding flow, empty states.
Phase 10 — Native workout tracking (future)	2–3 weeks	Build the native logging UI. Write through the same canonical tables. Add NativeAdapter.
Phase 11 — Mobile (future)	—	PWA-first (push notifications, offline logging, install banner). Or React Native + Expo.

9. Things to watch out for

Scope is large for v1. Eleven modules is a lot. Phases 1–4 alone give you a real, useful app — I’d strongly recommend shipping there and using it for a week before continuing. You’ll learn what matters in practice.

Lyfta API key UX trap. Lyfta shows the key once. If lost, regenerate → update the app. The setup screen will warn clearly and offer a ‘test connection’ button.

Strava token refresh. Strava access tokens expire every 6 hours. The adapter must auto-refresh using the refresh token. Easy, but a footgun if missed.

Strava Single Player Mode. Default is 1 athlete. Fine for personal use. If you ever share the app, file the form for an athlete-cap increase early (can take weeks).

MFP CSV is one-and-done. It’s an import, not sync. Going forward you’re logging in your app.

AI suggestions need guardrails. Without them, the model will tell you to eat 1,200 calories or deadlift 200kg. System prompt explicitly forbids: specific calorie targets unless a goal is set, weight prescriptions beyond +5% of recent lifts, any medical advice. Observation + general suggestion mode only.

Privacy. Encrypt API keys/tokens at rest. Supabase Row Level Security so users only see their own rows. Never log full request/response bodies in production.

10. Suggested additions worth considering

Workout readiness score. Combines yesterday's sleep + today's mood/energy + recent training load into a 'train hard, light, or rest?' signal. One AI call.

Photo-based meal logging. Gemini Flash is multimodal. Snap your plate → estimate. Not for precision, but great for quick logging.

Deload detector. If volume + intensity climb for 4+ weeks straight, surface a 'consider a deload' nudge.

Race goal tracker. With Strava in the mix — set a target race + date, AI suggests pace and weekly mileage progression.

Export everything. 'Download all my data as JSON/CSV' button. Future-you will thank present-you.

11. Open questions before we build

- Weight units — kg or lb?
- Calorie / macro targets — do you have set numbers, or want the AI to suggest based on goals?
- Where are you hosting code — GitHub? Self-hosted?
- Is this just for you, or will other people use it eventually? (Affects auth complexity, RLS rigor, Strava athlete-cap.)
- Phone OS — iPhone or Android? (Affects whether Apple Health is worth wiring up later instead of Google Fit.)
- Strava data depth — do you want full GPS routes / maps, or just summary stats?